

v++ Programming Language

Complete Technical Manual

Version 0.1.0

Language reference, compiler architecture, standard library, tooling, and build guide for the v0.1.0 release.

github.com/shauryaR790/V-

MIT License | August 2026

Table of Contents

1. Introduction and Philosophy
 2. Version 0.1.0 Release Summary
 3. Quick Start
 4. Complete Language Reference
 5. Type System
 6. Standard Library
 7. Projects and Modules
 8. Testing Framework
 9. Compiler Architecture
 10. Lexer Reference
 11. Parser and AST
 12. Type Checker
 13. Interpreter
 14. Native Codegen (LLVM)
 15. Runtime (C)
 16. Error Codes and Diagnostics
 17. CLI Reference
 18. Editor Extension
 19. Build System
 20. Repository Layout
 21. Examples Catalog
 22. Test Suite
 23. CI/CD
 24. Known Limitations (v0.1.0)
 25. Version History and Roadmap
 26. How This Was Built
 27. Contributing and Future Versions
-

1. Introduction and Philosophy

v++ (file extension .vpp, CLI binary vpp) is a teachable, practical programming language designed to feel approachable like Python while growing toward native performance and systems-level control like Rust or C++.

Design goals (v0.1.0)

- Readable syntax** - minimal punctuation, familiar control flow
 - Explicit types where they matter** - function signatures are written; local variables infer
-

- Real toolchain** - not a toy parser demo: lexer, parser, type checker, interpreter, optional native codegen, tests, projects, stdlib
- Teachable** - lesson examples, LANGUAGE.md, curriculum path
- Publishable** - MIT license, GitHub CI, install scripts, editor integration

What v++ is NOT (yet)

- Not a production systems language (no generics, no package registry, incomplete native codegen)
- Not self-hosting (compiler is written in Rust, not v++)
- Not memory-safe at the native layer yet (bootstrap runtime uses malloc without free)

Execution modes

Mode	Command	Requires
Interpreted (default)	vpp run file.vpp	Rust-built vpp only
Type-check only	vpp check file.vpp	Rust-built vpp only
Native compile	vpp build file.vpp	vpp built with --features codegen, LLVM, clang, MSVC on Windows

2. Version 0.1.0 Release Summary

Field	Value
Package name	vpp
Version	0.1.0
Edition	Rust 2021
License	MIT
Repository	https://github.com/shauryaR790/V-
Default run	Interpreter via tree-walking eval
Optional features	codegen (LLVM), lsp (language server)

Feature checklist (v0.1.0)

- (done) Lexer with spans and comment skipping
 - (done) Recursive-descent parser
 - (done) Local type inference on let
 - (done) Functions with explicit param/return types
 - (done) if / else, while, for over ranges and arrays
 - (done) break / continue
 - (done) Strings, arrays, + concatenation
 - (done) Struct definitions and struct literals
 - (done) Option<T>, Result<T,E>, Some, None, Ok, Err
 - (done) match expressions and statements
 - (done) Module imports with std path resolution
 - (done) test blocks, assert, assert_eq, vpp test
 - (done) vpp init, vpp.toml projects
 - (done) Standard library (math, io, string, assert stub)
 - (done) Formatter (vpp fmt)
 - (done) Numbered error codes (E0001-E0300)
 - (done) VS Code / Cursor extension with Run button
 - (done) GitHub Actions CI
 - (todo) Full native codegen (partial - see section 14)
 - (todo) Package manager
 - (todo) Generics / traits
-

3. Quick Start

Windows (recommended path)

```
git clone https://github.com/shauryaR790/V-.git
cd V-
.\setup.ps1
```

Then open examples\hello.vpp in Cursor and press F5.

Manual run

```
$env:Path = "$env:USERPROFILE\.cargo\bin;" + $env:Path  
.\vpp.ps1 run examples\hello.vpp
```

Create a project

```
.\vpp.ps1 init myapp  
cd myapp  
..\vpp.ps1 run  
..\vpp.ps1 test
```

4. Complete Language Reference

4.1 Comments

```
// This is a line comment. No block comments in v0.1.0.
```

4.2 Literals

Literal	Example	Type
Integer	42, -7	int
Float	3.14	float
Boolean	true, false	bool
String	"hello"	string
Array	[1, 2, 3]	array[T] inferred

String escapes: \n, \t, \\, \'

4.3 Variables

```
let x = 10           // inferred int
let name: string = "Alex" // explicit type
x = x + 1           // reassignment (must be declared first)
```

There is no mut keyword. Reassignment is allowed for bindings already declared with let in an enclosing scope.

4.4 Functions

```
fn add(a: int, b: int) -> int {
    return a + b
}

fn main() -> int {
    print(add(2, 3))
    return 0
}
```

Rules:

- All parameters require type annotations
- Return type is required
- Non-void functions must contain at least one return (type checker enforces)
- Top-level statements are allowed (no main required for scripts)

4.5 Control flow

if / else

```
if score >= 90 {
    print("A")
} else {
    print("B")
}
```

while

```
while n > 0 {  
    n = n - 1  
}
```

for (integer range)

Half-open range [start, end):

```
for i in 0..5 {  
    print(i)    // prints 0, 1, 2, 3, 4  
}
```

for (array iteration)

```
for item in [10, 20, 30] {  
    print(item)  
}
```

break and continue

```
for n in 1..10 {  
    if n == 3 { continue }  
    if n == 8 { break }  
    print(n)  
}
```

Only valid inside loops (type checker rejects at top level).

4.6 Operators

Precedence (low to high)	Operators
Assignment	=
Logical OR	\
Logical AND	&&
Equality	== !=
Comparison	< <= > >=
Additive	+ -
Multiplicative	* / %
Unary	! -
Postfix	() [] .

String concatenation uses + when both operands are strings.

4.7 Arrays

```
let nums = [1, 2, 3]
print(nums[0])
print(len(nums))
```

Empty arrays [] are rejected unless you could annotate (not yet supported as annotation-only empty literal).

All elements must have the same type.

4.8 Structs

```
struct Person {
    name: string
    age: int
}

let p = Person { name: "Alex", age: 20 }
print(p.name)
```

Field syntax in struct definition: no commas between fields (newline-separated).

Struct literal field syntax: field: value pairs, comma or newline separated.

4.9 Enums (parsed, rarely used in examples)

```
enum Color {  
    Red  
    Green  
    Blue  
}  
  
enum MaybeInt {  
    Some(int)  
    None  
}
```

User-defined enums parse and register; most examples use built-in Option and Result names.

4.10 Option and Result

Built-in generic-style types (no user generic syntax yet):

```
fn find() -> Option<int> {  
    return Some(42)  
    return None  
}  
  
fn divide(a: int, b: int) -> Result<int, string> {  
    if b == 0 {  
        return Err("division by zero")  
    }  
    return Ok(a / b)  
}
```

Some, None, Ok, Err are variant constructors. Type inference uses expected return type or assignment context.

4.11 match

```
match maybe {  
  Some(n) => {  
    print(n)  
  }  
  None => {  
    print(0)  
  }  
}
```

Patterns supported:

- `_` wildcard
- Literal patterns
- `Variant(binding)` or `Enum.Variant(binding)`
- Struct patterns `{ field: bind, ... }`

Match can be used as statement or expression. All arms should produce compatible types when used as expression.

Exhaustiveness is NOT compile-time checked in v0.1.0.

4.12 Modules

```
import "std/math.vpp"  
import "math.vpp"          // relative to current file directory
```

Imports are hoisted, resolved recursively, merged into one flat namespace.

4.13 Tests

```
test "addition works" {  
  assert_eq(add(2, 2), 4)  
}  
  
test "truth" {  
  assert(1 + 1 == 2)  
}
```

Run with `vpp test` inside a project.

4.14 Built-in functions

Name	Args	Returns	Description
print	any printable	void	Prints each arg + newline
len	array or string	int	Length
assert	bool	void	Fails if false
assert_eq	T, T	void	Fails if not equal

5. Type System

5.1 Internal types (src/types/mod.rs)

```
Int, Float, Bool, String
Array(Box<Type>)
Struct { name, fields: HashMap<String, Type> }
Enum { name, variants: HashMap<String, Vec<Type>> }
Option(Box<Type>)
Result { ok: Box<Type>, err: Box<Type> }
Function { params, ret }
Void, Error (internal)
```

5.2 Type annotations in source (TypeAnn)

```
int, float, bool, string
array[int]
Person // named struct
Option<int>
Result<int, string>
```

5.3 Inference rules

1. let x = expr - x gets type of expr
2. let x: T = expr - expr must match T
3. Function params - always explicit, no inference
4. None / Some(e) - needs expected Option<T> from context
5. Ok(e) / Err(e) - needs expected Result<T,E> from context
6. Integer literals - always int
7. Float literals - always float
8. Array literals - homogeneous; type is array[elem]
9. Binary ops - both sides must agree (numeric rules apply)
10. String + - both operands string

5.4 Name resolution

Single flat scope per function/block after imports merged. No modules namespaces. Shadowing in inner blocks via push/pop scope.

6. Standard Library

Location: std/ relative to project or compiler install.

std/math.vpp

Function	Signature
add	(int, int) -> int
abs	(int) -> int
max	(int, int) -> int
min	(int, int) -> int
pow	(int, int) -> int

std/io.vpp

Function	Signature
greet	(string) -> int
println	(string) -> int

std/string.vpp

Function	Notes
upper(s)	Placeholder - returns input unchanged
repeat(s, n)	Concatenates s n times

std/assert.vpp

Documentation module. Real assert / assert_eq are compiler builtins, not functions in this file.

Import example:

```
import "std/math.vpp"
print(add(2, 2))
```

7. Projects and Modules

7.1 vpp.toml

```
name = "myapp"
version = "0.1.0"
entry = "src/main.vpp"
```

Parser: line-based key = "value", # comments, unknown keys ignored.

7.2 Project layout (from vpp init)

```
myapp/  
  vpp.toml  
  src/  
    main.vpp  
  tests/  
    smoke.vpp
```

7.3 Import resolution algorithm

1. If path is relative (math.vpp, ./foo.vpp): look in importing file's directory
2. If path starts with std/: search std paths in order:
 - {project_root}/std
 - \$VPP_HOME/std
 - {compiler_exe_dir}/std
 - {compiler_source}/std (dev builds)
3. Add .vpp extension if missing
4. Recursive load; circular imports error
5. Imported items appended to program (flat merge)

8. Testing Framework

8.1 Test syntax

```
test "descriptive name" {  
  assert(condition)  
  assert_eq(actual, expected)  
}
```

8.2 vpp test algorithm

1. Find vpp.toml walking up from cwd
2. Collect all .vpp under tests/ (recursive, sorted)

3. If empty, use src/ tree
4. For each file: type-check, run all test blocks via interpreter
5. Print summary; exit 1 if any failure

8.3 Assertion semantics

- assert(false) -> runtime error "assertion failed"
 - assert_eq(a, b) -> compares runtime Value equality; prints both sides on failure
-

9. Compiler Architecture

9.1 Pipeline diagram

```

.vpp file(s)
  |
  v
+-----+
| Lexer  | src/lexer/   -> Vec<Token>
+-----+
  |
  v
+-----+
| Parser | src/parser/  -> Program (AST)
+-----+
  |
  v
+-----+
| Modules | src/modules/ resolve imports, merge
+-----+
  |
  v
+-----+
| TypeChecker | src/types/check.rs -> TypedProgram
+-----+
  |
  +-----+-----+-----+
  v             v             v
+-----+ +-----+ +-----+
| Interpreter | | Formatter | | LLVM codegen |
| src/interp | | src/fmt   | | src/codegen |
+-----+ +-----+ +-----+
      vpp run      vpp fmt      vpp build

```

9.2 Driver entry points (src/driver.rs)

Function	Purpose
parse(source)	Lex + parse only
check(source)	Parse + typecheck in-memory
check_path(path)	Load imports + typecheck file
run(source, path)	Typecheck + interpret
compile(...)	Typecheck + native codegen
run_tests_in_project(path)	Project test runner
init_project(dir, name)	Scaffold project
format_source(source)	Format string

10. Lexer Reference

File: src/lexer/mod.rs, src/lexer/token.rs

Token list

Keywords: let, fn, if, else, while, for, in, return, true, false, struct, enum, import, match, test, break, continue, Option, Result

Types: int, float, bool, string

Operators: + - * / % = == ! != < <= > >= && ||

Punctuation: () { } (todo) , : . .. -> =>

Literals: IntLit, FloatLit, StringLit, Ident

Other: Newline, Eof

Lexer errors

- E0001: invalid character
 - E0002: unterminated string
-

11. Parser and AST

File: src/parser/mod.rs, src/ast/mod.rs

Top-level items

Item ::= Import | Struct | Enum | Function | Test | Statement

Expression precedence

Documented in section 4.6. Parser uses Pratt-style layering in parse_expr.

Notable parse rules

- Struct literal prefix requires CapitalCase name before { (avoids match x { ambiguity)
- return optionally followed by expression
- test requires string literal name
- Range: `expr .. expr` only valid as for iterator

AST statement variants

Let, Expr, If, While, For, Return, Break, Continue, Match, Block

AST expression variants

Int, Float, Bool, String, Ident, Binary, Unary, Call, Index, Field, Array, StructLit, Range, Assign, Match

Every node carries Span for diagnostics.

12. Type Checker

File: `src/types/check.rs`

Phases

1. Register struct/enum types
2. Register function signatures (forward decl)
3. Check function bodies, tests, top-level stmts

TypedProgram output

```
functions: HashMap<String, FunctionInfo>
structs, enums
tests: Vec<TestInfo>
top_level: Vec<TypedStmt>
symbols: SymbolIndex (for LSP)
source_file: PathBuf
```

For-loop lowering

- for v in start..end -> TypedStmt::ForInt { start, end } (half-open)
- for v in arr -> TypedStmt::ForArray { array, elem_ty }

Loop depth tracking

break and continue increment/decrement loop_depth; error if used outside loop.

13. Interpreter

File: src/interp/mod.rs

Value representation

```
Int(i64), Float(f64), Bool(bool)
String(Rc<String>)
Array(Rc<Vec<Value>>)
Struct { name, fields: HashMap }
Variant { enum_name, variant, payload: Vec<Value> }
Void
```

Execution model

- Tree-walking eval over TypedStmt / TypedExpr
- Lexical scopes: Vec<HashMap<String, Value>>
- Function calls: new scope, bind params, exec body
- Return: returning flag stops execution
- Break/continue: flags consumed by innermost loop

Builtins implemented

print, len, assert, assert_eq + user functions

Match semantics

First matching arm wins. No exhaustiveness check. Wildcard always matches.

Known interpreter quirks

- Missing return in non-void function -> returns Int(0) at runtime (should be caught by type checker)
- Complex literal patterns in match may fail at runtime

14. Native Codegen (LLVM)

Feature flag: codegen

Files: src/codegen/llvm.rs, src/codegen/llvm_stubs.c, build.rs

Pipeline

1. Build LLVM module via inkwell
2. Emit main() that runs top-level statements
3. Write IR to temp .ll
4. clang -c out.ll -o out.o
5. clang -c vpp_runtime.c -o vpp_runtime.o
6. clang out.o vpp_runtime.o -o executable

Type mapping

v++	LLVM
int	i64
float	f64
bool	i1
string	i8* (global string ptr - ABI mismatch with runtime)
array	VppArray* via runtime

Implemented in codegen

Literals, locals, binary/unary ops, if/while, for-int, for-array, arrays, index, assign, print (int/float/bool/string), len, user function calls

NOT implemented (errors directing to vpp run)

match, break, continue, struct field/literal, Option/Result, assert, string concat

Windows LLVM linking

Prebuilt LLVM-C.dll lacks target init symbols. llvm_stubs.c provides no-op stubs for LLVM_Initialize* symbols. Built via build.rs with MSVC /INCLUDE: directives.

Known native bug (v0.1.0)

String printing passes i8* to vpp_print_str(VppString*) - causes access violation. Use interpreter for string-heavy programs until fixed in v0.2.0.

15. Runtime (C)

File: runtime/vpp_runtime.c

Structures

```
VppString { char* data; int64_t ref_count; }
VppArray { void* data; int64_t len; elem_size; ref_count; }
```

Exported functions

vpp_print_int, vpp_print_float, vpp_print_bool, vpp_print_str
vpp_alloc, vpp_string_new/retain/release

vpp_make_array, vpp_array_len, vpp_array_data, vpp_array_retain/release
vpp_strlen

Memory model (bootstrap)

Uses malloc. ARC helpers exist for strings/arrays but full integration pending. Documented as temporary.

16. Error Codes and Diagnostics

File: src/error.rs - uses thiserror + miette fancy diagnostics

Code	Name	When
E0001	InvalidCharacter	Bad char in lexer
E0002	UnterminatedString	Unclosed "
E0003	UnexpectedToken	Parser mismatch
E0004	UnexpectedEof	Parser expected more
E0100	TypeMismatch	Type error
E0101	WrongArgCount	Call arity
E0102	EmptyArrayNoType	[] without elements
E0103	ArrayElementMismatch	Mixed array types
E0104	ImmutableAssign	Assign to undeclared name
E0105	MissingReturn	Function missing return
E0106	InvalidForIter	for over non-array/range
E0200	UndefinedVariable	Unknown ident
E0201	UndefinedFunction	Unknown call
E0300	Codegen	Native compile error
-	Io	File errors
-	Other	Imports, tests, runtime

CLI attaches source text via with_source() for caret display.

17. CLI Reference

Binary: vpp (src/main.rs, clap)

Command	Usage
check	vpp check file.vpp
run	vpp run [file.vpp] - no file uses project entry
build	vpp build [file] [-o exe] - needs codegen feature
compile	vpp compile file.vpp [-o file.ll] - emit LLVM IR
fmt	vpp fmt file.vpp - in-place format
test	vpp test [path]
init	vpp init [name] [-p dir]
lsp	vpp lsp - needs --features lsp

Wrappers: vpp.ps1, vpp.cmd, setup.ps1, install.ps1

LSP binary: vpls (separate, feature-gated)

18. Editor Extension

Path: editor/vscode-vpp/

Version: 0.3.0 (independent of compiler 0.1.0)

Features

- Syntax highlighting (TextMate grammar)
- Language configuration (brackets, comments)
- Icon theme for .vpp files
- Commands: Run File, Check File, Run Tests
- Keybindings: F5, Ctrl+Shift+R
- Editor title Run button (play icon)

Compiler discovery order

1. vpp.compilerPath setting

2. vpp.ps1 in workspace root
3. vpp.cmd
4. target/release/vpp.exe or target/debug/vpp.exe

Not wired in v0.1.0

LSP auto-start from extension (must run vppls separately)

19. Build System

Cargo features

Feature	Enables
default	Interpreter + check + fmt only
codegen	inkwell + llvm-sys + native compile
lsp	tower-lsp + tokio + vppls binary

Dependencies

- clap 4, thiserror 2, miette 7 (always)
- inkwell (git), llvm-sys 221 (codegen)
- tower-lsp 0.20, tokio 1 (lsp)

Build commands

```
# Interpreter only (default)
cargo build --release

# With native codegen
$env:LLVM_SYS_221_PREFIX = "C:\Program Files\LLVM"
cargo build --release --features codegen

# Language server
cargo build --release --features lsp --bin vppls
```

Platform requirements

Platform	Interpreter	Native codegen
Windows	Rust + cargo	+ LLVM 22 + clang + MSVC Build Tools
Linux	Rust + cargo	+ LLVM + clang
macOS	Rust + cargo	+ LLVM + clang

20. Repository Layout

```
V-/
src/           Rust compiler source
runtime/       C runtime for native builds
std/           v++ standard library
examples/      Demo and lesson programs
tests/         Rust integration tests
editor/        VS Code extension
docs/          Documentation (this manual)
scripts/       PDF generator, utilities
.github/       CI workflows
Cargo.toml     Rust package manifest
build.rs       LLVM stub linking
vpp.ps1        Windows launcher
setup.ps1      One-command setup
LANGUAGE.md    Quick language reference
README.md      Project overview
LICENSE        MIT
```

21. Examples Catalog

File	Demonstrates
hello.vpp	print, let, int/string/bool
lesson01_basics.vpp	arithmetic, string concat
lesson02_functions.vpp	import, fn, if, main()
lesson03_loops.vpp	for, break, continue, arrays
arrays.vpp	array ops, len, loops
fib.vpp	recursion
structs.vpp	struct def, literal, fields
option_result.vpp	Option, Result, match
match_test.vpp	fn returning Result
math.vpp	local module
modules_main.vpp	relative import

22. Test Suite

Rust integration tests (tests/)

- lexer.rs - tokens, operators, range
- parser.rs - let, fn, if, for, arrays
- typecheck.rs - inference, error codes
- end_to_end.rs - example typechecks
- test_runner.rs - smoke.vpp tests, init scaffold

In-module unit tests

lexer, parser, interp, fmt, project modules

Running

```
cargo test
```

23. CI/CD

File: .github/workflows/ci.yml

On push/PR to main:

- cargo test --all-targets
 - Run/check lesson examples
 - No codegen or LSP in CI (yet)
-

24. Known Limitations (v0.1.0)

Language

- No generics, traits, or interfaces
- No user void type
- No block comments
- No string indexing
- No package manager
- Flat import namespace
- Match not exhaustiveness-checked
- Enum support minimal

Interpreter

- Default int(0) on missing return (partially caught by checker)
-

Codegen

- Most language features unimplemented (see section 14)
- String ABI bug causes native crash on hello.vpp

Tooling

- Formatter is basic token-based
- LSP diagnostics lack precise spans in editor
- Grammar highlighting missing some keywords

Runtime

- malloc without free (bootstrap)
-

25. Version History and Roadmap

v0.1.0 (current - August 2026)

Initial public release. Interpreter-complete language core. Project system, tests, stdlib, extension, CI.

Planned v0.2.0

- Fix native string ABI / hello.vpp native build
- Codegen: structs, match, break/continue
- LSP wired into extension with live diagnostics
- Improved formatter
- GitHub Release binaries (prebuilt vpp.exe)

Planned v0.3.0

- Package manager (git deps): vpp add
 - std/fs, std/json modules
 - Error message improvements (did you mean)
-

Planned v0.4.0

- Generics syntax
- Trait/interface system (design TBD)

Planned v1.0.0

- Stable language spec (SPEC.md)
- Semver guarantee for language and CLI
- Full native codegen parity with interpreter
- Curriculum (20+ lessons)
- Playground website
- Production memory model (ARC default)

Version numbering policy (future)

- MAJOR** - breaking language or CLI changes
- MINOR** - new features, backward compatible
- PATCH** - bug fixes

Document all changes in CHANGELOG.md starting v0.2.0.

26. How This Was Built

Timeline and approach

v++ was implemented as a Rust crate in incremental phases:

1. Lexer + parser + AST - recursive descent, span tracking
 2. Type checker - local inference, explicit functions
 3. Interpreter - tree-walk eval for fast iteration
 4. Structs, match, Option/Result - extended AST + checker + interp together
 5. Modules - import graph, std paths
 6. Projects + tests - vpp.toml, test blocks, assert builtins
 7. Tooling - fmt, CLI, setup scripts, extension
 8. Codegen (partial) - inkwell IR + clang + C runtime
 9. Publish - docs, CI, GitHub, PDF manual
-

Why Rust for the compiler

- Memory safety while building infrastructure
- Excellent tooling (cargo test, clap, miette)
- inkwell for LLVM bindings
- tower-lsp for language server

Why interpreter-first

Native codegen is expensive to complete. Interpreter lets language design iterate quickly. vpp run works without LLVM installed.

Key design decisions

Decision	Rationale
Flat imports	Simplicity for teaching
Half-open ranges	Matches Python-style range(n) intuition
Explicit fn types	Teaching types without full inference
Builtins not keywords	Easier to add stdlib later
test blocks in source	Rust-inspired; no external test syntax

27. Contributing and Future Versions

Repository

<https://github.com/shauryaR790/V->

How to add a language feature (checklist)

1. Lexer token (if keyword) - src/lexer/
2. Parser rule - src/parser/mod.rs
3. AST node - src/ast/mod.rs
4. Typed AST - src/types/mod.rs

5. Type checker - src/types/check.rs
6. Interpreter - src/interp/mod.rs
7. Codegen (optional) - src/codegen/llvm.rs
8. Example + test
9. Update LANGUAGE.md and this manual
10. Bump version in Cargo.toml + CHANGELOG

Documentation updates per version

When releasing v0.2.0+:

- Copy this manual to VPP_COMPLETE_MANUAL_v0.2.0.md
 - Update section 2, 24, 25 with delta
 - Regenerate PDF: python scripts/build_manual_pdf.py
 - Tag git: git tag v0.2.0
-

Appendix A: Public Rust API

From src/lib.rs:

```
check, check_file, check_path, check_with_index
parse, run, compile, emit_ir, format_source
run_tests_in_project, init_project, project_entry
CompileOptions, Manifest, find_project_root, load_manifest
VppError, VppResult
ensure_llvm_stubs_linked (codegen)
```

Appendix B: Symbol Index (LSP)

src/symbols.rs tracks Struct, Enum, Variant, Function, Variable defs with file + span for go-to-definition when LSP feature enabled.

Appendix C: Formatter Algorithm

src/fmt/mod.rs - token-based re-indentation. Not a full pretty-printer. Inserts newlines after {, indents block contents, preserves string contents.

End of v++ Complete Manual v0.1.0

Generated for the V- repository. MIT License.